useMemo Hook: Complete Interview Guide

1. What is useMemo?

> **Definition (say this in an interview):**
>
> `useMemo` is a React hook that **caches (memoizes) the result of a calculation between re-renders**. It only recalculates when one of its dependencies has changed; otherwise it returns the previously stored result.

```
const value = useMemo(() => calculate(a, b), [a, b]);
//                    ↑ function to run      ↑ dependencies
```

The core idea: a React component function re-runs from top to bottom on *every* render. Anything slow inside it also re-runs every time, even if its inputs didn't change. `useMemo` lets React skip that work.

2. Why is it used?

There are two real reasons:

1. **Skip expensive calculations** (heavy loops, filtering or sorting big lists).
2. **Keep a stable reference** for objects and arrays, so child components wrapped in `React.memo` don't re-render needlessly (explained in Section 5).

3. Your example WITHOUT useMemo

```
const expensiveValue = expensiveTask(input);
```

State: `count` and `input`. Two things can trigger a re-render.

```
RENDER 1 (page loads)
 └─ expensiveTask(0) runs ........ 🐢 SLOW (100M loop)

USER CLICKS "Increment"  → count: 0 → 1
RENDER 2
 └─ expensiveTask(0) runs AGAIN .. 🐢 SLOW  ← wasted! input didn't change

USER CLICKS "Increment"  → count: 1 → 2
RENDER 3
 └─ expensiveTask(0) runs AGAIN .. 🐢 SLOW  ← wasted again!
```

Result: the button feels laggy, even though it has nothing to do with the expensive value.

4. Your example WITH useMemo

```
const expensiveValue = useMemo(() => expensiveTask(input), [input]);
```

Think of `useMemo` as a **small box with one slot** that stores `[last dependencies, last result]`.

```
RENDER 1 (page loads)
 Box: empty
 └─ deps [0] → nothing stored → RUN expensiveTask(0) 🐢
 Box: { deps: [0], result: 0 }

USER CLICKS "Increment"  → count changes
RENDER 2
 └─ compare deps: [0] vs [0] → SAME ✅
 └─ return stored result instantly ⚡ (function NOT called)

USER TYPES 5
RENDER 3
 └─ compare deps: [5] vs [0] → DIFFERENT ❌
 └─ RUN expensiveTask(5) 🐢
 Box: { deps: [5], result: 10 }   ← old value overwritten

USER TYPES 0 again
RENDER 4
 └─ compare deps: [0] vs [5] → DIFFERENT ❌
 └─ RUN expensiveTask(0) again 🐢   ← it forgot the old 0
```

That last step is your note: the box has **one slot**, so it only remembers the most recent input.

	Without useMemo	With useMemo
Increment click	Slow	Instant
Input changes	Slow	Slow (needed)
Function runs	Every render	Only when deps change

5. Second example (easier, very common in interviews)

Example A: Search filter plus a dark-mode toggle

```
function ProductList({ products }) {
  const [query, setQuery] = useState("");
  const [dark, setDark] = useState(false);

  const filtered = useMemo(() => {
    console.log("Filtering...");
    return products.filter((p) =>
      p.name.toLowerCase().includes(query.toLowerCase())
    );
  }, [products, query]);

  return (
    <div className={dark ? "dark" : "light"}>
      <button onClick={() => setDark(!dark)}>Toggle theme</button>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
```

```
      {filtered.map((p) => <div key={p.id}>{p.name}</div>)}
    </div>
  );
}
```

```
Toggle theme  → dark changes, query/products same → filter SKIPPED ⚡
Type "phone"  → query changes                     → filter RUNS 🐢
```

With 10,000 products, toggling the theme would otherwise re-filter all 10,000 for no reason.

Example B: Referential equality (the second use)

```
const Child = React.memo(function Child({ style }) {
  console.log("Child rendered");
  return <div style={style}>Hi</div>;
});

function Parent() {
  const [count, setCount] = useState(0);

  // ❌ New object every render → Child re-renders every time
  // const style = { color: "red" };

  // ✅ Same object reference kept → Child skipped
  const style = useMemo(() => ({ color: "red" }), []);

  return (
    <>
      <button onClick={() => setCount(count + 1)}>{count}</button>
      <Child style={style} />
    </>
  );
}
```

Why: in JavaScript, `{} === {}` is `false`. Every render creates a *new* object, so `React.memo` thinks the prop changed. `useMemo` returns the **same reference**, so the child is skipped.

6. Interview Questions with Follow-ups

Q1. What is useMemo?

Answer: A hook that memoizes the result of a calculation. It runs your function, stores the result, and on later renders returns the stored result unless a dependency changed.

- **Follow-up: What does “memoize” mean?**
  Storing the result of a function call for given inputs, so the same inputs don’t need recomputation. It’s caching for function results.
- **Follow-up: Does it run on the first render?**
  Yes. The first render always runs the function, because there’s nothing in the cache yet.

Q2. Why and when would you use it?

Answer: Two cases: (1) a calculation is genuinely expensive, like filtering or sorting large data, and (2) you need a stable object or array reference to pass to a `React.memo` child or to a dependency array of another hook.

- **Follow-up: How do you decide something is "expensive"?**
  Measure it. Use `console.time` around the calculation or the React DevTools Profiler. A common rule of thumb is that if it takes about 1ms or more, consider memoizing it. Don't guess.
- **Follow-up: Would you use it for `num * 2`?**
  No. The cost of comparing dependencies and storing the cache is more than the calculation itself. Memoizing cheap work is a net loss.

Q3. How does the dependency array work?

Answer: React compares each dependency from the current render with the previous render using `Object.is` (like `===`). If all are equal, it returns the cached value. If any differ, it re-runs the function and stores the new result.

- **Follow-up: What if the array is empty `[]`?**
  The function runs once on mount and the value is never recalculated (until the component unmounts and remounts).
- **Follow-up: What if you pass no array at all?**
  It recalculates on every render, so memoization does nothing.
- **Follow-up: What happens if you pass an object as a dependency?**
  If that object is recreated every render, `Object.is` sees a new reference each time and the cache is useless. You'd need to memoize that object too, or depend on its primitive fields.

Q4. Does useMemo remember all previous inputs?

Answer: No. It stores only the **last** dependencies and result. If `input` goes 5 → 8 → 5, it recalculates for 5 the second time.

- **Follow-up: How would you cache every input?**
  Build your own cache, such as a `Map` or object stored in a `useRef`, or use a memoization utility like lodash's `memoize`. Be careful about unbounded memory growth.

Q5. useMemo vs useCallback?

Answer: `useMemo` caches a **value** (the return value of a function). `useCallback` caches the **function itself**.

```
useCallback(fn, deps)  ≡  useMemo(() => fn, deps)
```

- **Follow-up: When is useCallback useful?**
  When passing a function as a prop to a `React.memo` child, or when the function is a dependency of `useEffect`. A new function is created each render, so without `useCallback` the child always sees a "new" prop.

Q6. useMemo vs React.memo?

Answer: `useMemo` is a hook that caches a *value inside* a component. `React.memo` is a higher-order component that skips re-rendering a *whole component* if its props haven't changed.

- **Follow-up: How do they work together?**
  `React.memo` compares props by reference. `useMemo` and `useCallback` keep object and function props stable, so `React.memo` can actually skip renders. Without stable props, `React.memo` is defeated.

Q7. Does useMemo prevent re-renders?

Answer: No. The component still re-renders. `useMemo` only skips the *calculation inside* it.

- **Follow-up: Then how do you prevent the re-render?**
  Wrap the child in `React.memo`, or restructure state (move state down into a smaller component, or lift content up as `children`).

Q8. Is useMemo a guaranteed cache?

Answer: No. It's a **performance optimization, not a semantic guarantee**. React may discard cached values (for example, to free memory), so your code must still work correctly if the function re-runs.

- **Follow-up: What does that mean for how you write the function?**
  It must be **pure**: same inputs give the same output, with no side effects such as API calls, subscriptions, or mutations.
- **Follow-up: Why does my function run twice in development?**
  React Strict Mode intentionally double-invokes it in development to help detect impure code. It doesn't happen in production.

Q9. Can you use useMemo for side effects or API calls?

Answer: No. Use `useEffect` for side effects. `useMemo` runs during rendering and must be pure.

- **Follow-up: Can you store derived data using useEffect + useState instead?**
  You can, but it's worse. It causes an extra render and temporarily shows stale data. For values *derived* from props or state, calculate during render (with `useMemo` if expensive).

Q10. Should you wrap everything in useMemo?

Answer: No. It adds memory use and comparison overhead, and makes code harder to read. Use it where you've measured a problem or need reference stability.

- **Follow-up: Is there any tool that does it automatically?**
  Yes, the React Compiler can automatically memoize many values and functions at build time, reducing the need to write `useMemo` manually.

Q11. Where can you not call useMemo?

Answer: Like all hooks, only at the top level of a component or custom hook. Not inside loops, conditions, or nested functions.

- **Follow-up: Why?**
  React tracks hooks by **call order**. If the order changes between renders, React matches state and caches to the wrong hooks.

Q12. In your project, how did you demonstrate useMemo?

Answer: I had a slow function (100M-iteration loop) and two states, `count` and `input`. Without `useMemo`, clicking Increment re-ran the slow function every time and lagged. With `useMemo(() => expensiveTask(input), [input])`, the function ran only when `input` changed, so Increment became instant.

- **Follow-up: How did you verify it worked?**
  I placed a `console.log` inside `expensiveTask`. It printed only when the input changed, not when the button was clicked.

7. Quick Cheat Sheet

Concept	One-liner
Purpose	Cache a computed value between renders
Returns	The cached or newly computed **value**
Recalculates when	Any dependency changes (`Object.is`)
Cache size	One slot (last result only)
Prevents re-render?	No, only skips the calculation
Must the function be pure?	Yes
Use it for	Expensive calculations, stable object/array references
Don’t use it for	Cheap calculations, side effects, API calls